

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO5 – Precision (BL4)	Assessment:	Pseudocode Writing task
Weightage:	30%	Total Marks:	100
CO5 Statement:	Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.		
Student Name:	S lakshmi priya	Reg. No.:	192524284
Section / Batch:	AI&DS	Date:	26/08/2026

Instructions to Students

- * Identify the relevant concepts of DMA-based data transfer, vectored interrupts, I/O mapping techniques, bus arbitration, and hybrid I/O communication.
- * Analyze the requirements of different I/O devices by considering CPU involvement, interrupt priority, latency, bandwidth, throughput, and transfer speed.
- * Apply appropriate I/O techniques such as DMA, interrupt-driven I/O, vectored interrupts, memory-mapped I/O, I/O-mapped I/O, and bus arbitration to develop efficient solutions.
- * Develop pseudocode algorithms for DMA data transfer, prioritized interrupt handling, dynamic I/O mapping selection, bus arbitration, and hybrid polling-DMA communication.
- * Evaluate the proposed I/O mechanisms by interpreting CPU utilization, data transfer efficiency, interrupt response time, communication performance, resource allocation, and overall system suitability.

QUESTIONS

1. A real-time embedded system receives interrupt requests from sensors, communication modules, and storage devices at unpredictable intervals. Develop pseudocode for a nested interrupt handling mechanism that allows a higher-priority interrupt to pre-empt a lower-priority service routine. The algorithm should save the current processor context, service the higher-priority interrupt, restore the interrupted routine, and ensure that pending interrupts are handled correctly.
2. A data-intensive embedded system must transfer a large amount of information between memory and multiple I/O devices without continuously occupying the processor. Write pseudocode for a scatter-gather DMA mechanism that transfers data stored in multiple non-contiguous memory locations. The algorithm should maintain a list of memory descriptors, initiate successive transfers, update descriptor status, and handle completion and error conditions.
3. A computer system supports several I/O devices that generate data at different rates, causing temporary variations in the amount of data waiting for processing. Design pseudocode for an I/O buffering mechanism that manages incoming and outgoing data using appropriate buffer allocation and release strategies. The algorithm should monitor buffer occupancy, handle buffer-full and buffer-empty conditions, and prevent data loss while maintaining efficient device communication.
4. A high-speed embedded platform contains multiple communication devices competing for shared system resources. Develop pseudocode for a priority-based resource allocation mechanism that assigns communication resources according to device urgency and bandwidth requirements. The algorithm should dynamically update priorities, allocate resources to eligible devices, release resources after transmission, and prevent low-priority devices from being indefinitely delayed.
5. A real-time system must transfer data from several peripheral devices while meeting strict timing constraints. Write pseudocode for a real-time I/O scheduling mechanism that determines when each device should perform data transfers based on deadlines, transfer duration, priority, and available system bandwidth. The algorithm should schedule time-critical transfers first, monitor missed deadlines, and reschedule pending requests to improve overall system responsiveness.

Code of Conduct

I s Lakshmi Priya(192524284)____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: s priya_____

```

1  Pseudocode: Nested Interrupt Handling
2  GLOBAL:
3      interrupt_stack = empty stack
4      current_priority = LOWEST_PRIORITY
5      pending_interrupts = priority queue
6
7  ON_INTERRUPT(interrupt):
8      if interrupt.priority <= current_priority then
9          // Lower/equal priority: do not pre-empt current ISR
10         pending_interrupts.insert(interrupt)
11         return
12     end if
13
14     // Higher-priority interrupt: pre-empt current execution
15     SAVE_CONTEXT()
16     // Save PC, processor status/registers, stack pointer, etc.
17     end SAVE_CONTEXT
18
19     PUSH interrupt context onto interrupt_stack
20
21     current_priority = interrupt.priority
22
23     SERVICE_INTERRUPT(interrupt)
24
25     // Restore the previously interrupted routine
26     POP previous context from interrupt_stack
27     RESTORE_CONTEXT(previous context)
28
29     current_priority = previous context.priority
30
31     // Check whether another interrupt is waiting
32     HANDLE_PENDING_INTERRUPTS()
33
34
35 HANDLE_PENDING_INTERRUPTS():
36     while pending_interrupts is not empty do
37
38         next_interrupt = pending_interrupts.highest_priority()
39
40         if next_interrupt.priority > current_priority then
41             remove next_interrupt from pending_interrupts
42
43             SAVE_CONTEXT()
44             PUSH current context onto interrupt_stack
45
46             current_priority = next_interrupt.priority
47             SERVICE_INTERRUPT(next_interrupt)
48
49             POP previous context from interrupt_stack
50             RESTORE_CONTEXT(previous context)
51
52             current_priority = previous context.priority
53         else
54             break
55         end if
56     end while
57
58
59 SERVICE_INTERRUPT(interrupt):

```

```

60  acknowledge_interrupt(interrupt)
61
62  switch interrupt.source:
63      case SENSOR:
64          process_sensor_data()
65      case COMMUNICATION:
66          process_communication()
67      case STORAGE:
68          process_storage_request()
69      default:
70          handle_unknown_interrupt()
71  end switch
72
73  clear_interrupt_flag(interrupt)

```

How it works

1. When an interrupt occurs, its priority is compared with the priority of the currently executing routine.
2. Higher-priority interrupts pre-empt the current ISR. The processor context is saved on `interrupt_stack`.
3. The higher-priority ISR executes.
4. After completion, the saved context is restored, allowing the interrupted ISR to continue exactly where it stopped.
5. Lower- or equal-priority interrupts are placed in `pending_interrupts` rather than causing another pre-emption.
6. After an ISR finishes, pending interrupts are checked in highest-priority-first order.
7. The stack allows multiple levels of nesting, e.g. a low-priority ISR can be interrupted by a medium-priority ISR, which can itself be interrupted by a high-priority ISR.

This mechanism ensures that urgent real-time events receive immediate CPU attention while lower-priority interrupt processing is resumed without losing its execution state.

ChatGPT is AI and can make mistakes.

Chat with ChatGPT

2 Pseudocode: Scatter-Gather DMA

STRUCT DMA_DESCRIPTOR:

```

source_address
destination_address
transfer_length
status          // PENDING, ACTIVE, COMPLETE, ERROR
error_code
next_descriptor

```

GLOBAL:

```

descriptor_list
current_descriptor
dma_active = FALSE

```

INITIALIZE_DMA_TRANSFER(memory_blocks, io_device):

```

// Create descriptors for all non-contiguous memory blocks
descriptor_list = empty list

```

```

for each block in memory_blocks do
    descriptor = new DMA_DESCRIPTOR

```

```

    descriptor.source_address = block.address
    descriptor.destination_address = io_device.address
    descriptor.transfer_length = block.length
    descriptor.status = PENDING
    descriptor.error_code = NONE

```

```

    append descriptor to descriptor_list
end for

// Link descriptors for successive DMA transfers
for i = 0 to length(descriptor_list) - 2 do
    descriptor_list[i].next_descriptor = descriptor_list[i + 1]
end for

last descriptor_list element.next_descriptor = NULL

current_descriptor = descriptor_list[0]

START_NEXT_TRANSFER()

```

START_NEXT_TRANSFER():

```

if current_descriptor == NULL then
    dma_active = FALSE
    SIGNAL_TRANSFER_COMPLETE()
    return
end if

if current_descriptor.status == COMPLETE then
    current_descriptor = current_descriptor.next_descriptor
    START_NEXT_TRANSFER()
    return
end if

current_descriptor.status = ACTIVE
dma_active = TRUE

// Program DMA controller
DMA.source    = current_descriptor.source_address
DMA.destination = current_descriptor.destination_address
DMA.length    = current_descriptor.transfer_length

DMA.enable_interrupt_on_completion()
DMA.enable_interrupt_on_error()

DMA.start()

```

DMA_INTERRUPT_HANDLER():

```

if DMA.error_detected() then

    current_descriptor.status = ERROR
    current_descriptor.error_code = DMA.get_error_code()

    DMA.stop()
    dma_active = FALSE

    HANDLE_DMA_ERROR(current_descriptor)
    return
end if

if DMA.transfer_complete() then

```

```

current_descriptor.status = COMPLETE

// Update to the next non-contiguous memory block
current_descriptor = current_descriptor.next_descriptor

if current_descriptor != NULL then
    START_NEXT_TRANSFER()
else
    dma_active = FALSE
    SIGNAL_TRANSFER_COMPLETE()
end if
end if

```

HANDLE_DMA_ERROR(descriptor):

```

LOG_ERROR(descriptor.error_code)

if ERROR_IS_RECOVERABLE(descriptor.error_code) then

    descriptor.status = PENDING

    RESET_DMA_CONTROLLER()

    // Retry the failed descriptor
    START_NEXT_TRANSFER()

else
    for each descriptor in descriptor_list do
        if descriptor.status == ACTIVE or
           descriptor.status == PENDING then
            descriptor.status = ERROR
        end if
    end for

    SIGNAL_TRANSFER_ERROR()
end if

```

Operation

- Each non-contiguous memory region is represented by a DMA descriptor containing its address, transfer length, status, and pointer to the next descriptor.
- The CPU initializes the descriptor list and starts the DMA controller; it is not required to copy the data itself.
- When one transfer completes, the DMA interrupt handler marks its descriptor COMPLETE and starts the next descriptor.
- If an error occurs, the descriptor is marked ERROR, the error code is recorded, and the transfer is either retried or terminated depending on whether the error is recoverable.
- Once the final descriptor is completed, the DMA controller signals overall transfer completion.

This allows large data sets spread across multiple non-contiguous memory locations to be transferred efficiently while minimizing continuous processor involvement.

1. 3 ChatGPT said:

Pseudocode: I/O Buffering Mechanism

CONSTANT BUFFER_SIZE = N

STRUCT BUFFER:

```

data[BUFFER_SIZE]
read_index = 0
write_index = 0
count = 0

```

GLOBAL:

input_buffer
output_buffer

INITIALIZE():

input_buffer = allocate BUFFER
output_buffer = allocate BUFFER

input_buffer.read_index = 0
input_buffer.write_index = 0
input_buffer.count = 0

output_buffer.read_index = 0
output_buffer.write_index = 0
output_buffer.count = 0

// Called when an I/O device produces new data

INPUT_DATA(data):

if input_buffer.count == BUFFER_SIZE then
 HANDLE_BUFFER_FULL(data)
 return
end if

input_buffer.data[input_buffer.write_index] = data

input_buffer.write_index =
 (input_buffer.write_index + 1) MOD BUFFER_SIZE

input_buffer.count = input_buffer.count + 1

SIGNAL_DATA_AVAILABLE()

// Called by the processor/application to obtain input data

GET_INPUT_DATA():

if input_buffer.count == 0 then
 HANDLE_BUFFER_EMPTY()
 return NO_DATA
end if

data = input_buffer.data[input_buffer.read_index]

input_buffer.read_index =
 (input_buffer.read_index + 1) MOD BUFFER_SIZE

input_buffer.count = input_buffer.count - 1

SIGNAL_BUFFER_SPACE_AVAILABLE()

return data

// Called when the processor has data to send to an output device

QUEUE_OUTPUT(data):

if output_buffer.count == BUFFER_SIZE then

```
    HANDLE_OUTPUT_BUFFER_FULL(data)
    return BUFFER_FULL
end if
```

```
output_buffer.data[output_buffer.write_index] = data
```

```
output_buffer.write_index =
    (output_buffer.write_index + 1) MOD BUFFER_SIZE
```

```
output_buffer.count = output_buffer.count + 1
```

```
SIGNAL_OUTPUT_DEVICE()
```

```
return SUCCESS
```

```
// Called when the output device is ready for more data
```

```
SEND_OUTPUT():
```

```
if output_buffer.count == 0 then
    HANDLE_OUTPUT_BUFFER_EMPTY()
    return NO_DATA
end if
```

```
data = output_buffer.data[output_buffer.read_index]
```

```
output_buffer.read_index =
    (output_buffer.read_index + 1) MOD BUFFER_SIZE
```

```
output_buffer.count = output_buffer.count - 1
```

```
DEVICE_SEND(data)
```

```
SIGNAL_BUFFER_SPACE_AVAILABLE()
```

```
return SUCCESS
```

```
HANDLE_BUFFER_FULL(data):
```

```
// Prevent overwriting existing data
if CAN_WAIT_FOR_BUFFER_SPACE() then
    WAIT until input_buffer.count < BUFFER_SIZE
    INPUT_DATA(data)
else
    STORE_IN_SECONDARY_BUFFER(data)
    if SECONDARY_BUFFER_FULL() then
        SIGNAL_OVERFLOW_ERROR()
    end if
end if
```

```
HANDLE_BUFFER_EMPTY():
```

```
// No input data is available
WAIT until input_buffer.count > 0
return
```

```
HANDLE_OUTPUT_BUFFER_FULL(data):
```

```

// Prevent loss of data when output device is slow
if CAN_WAIT_FOR_BUFFER_SPACE() then
    WAIT until output_buffer.count < BUFFER_SIZE
    QUEUE_OUTPUT(data)
else
    STORE_IN_SECONDARY_BUFFER(data)
    if SECONDARY_BUFFER_FULL() then
        SIGNAL_OUTPUT_OVERFLOW_ERROR()
    end if
end if
end if

```

HANDLE_OUTPUT_BUFFER_EMPTY():

```

// Output device has no data to transmit
WAIT until output_buffer.count > 0
return

```

MONITOR_BUFFERS():

```

periodically do

    input_occupancy =
        (input_buffer.count / BUFFER_SIZE) * 100

    output_occupancy =
        (output_buffer.count / BUFFER_SIZE) * 100

    if input_occupancy >= 80 then
        ENABLE_INPUT_PROCESSING_PRIORITY()
    end if

    if output_occupancy >= 80 then
        ENABLE_OUTPUT_TRANSMISSION_PRIORITY()
    end if

    if input_occupancy <= 20 then
        NORMALIZE_INPUT_PROCESSING()
    end if

    if output_occupancy <= 20 then
        NORMALIZE_OUTPUT_TRANSMISSION()
    end if

end periodically

```

How it works

- Buffer allocation: Separate circular buffers are allocated for incoming and outgoing data.
- Buffer occupancy: count tracks the number of occupied entries, while read_index and write_index identify where data is removed and inserted.
- Buffer-full condition: New data is not allowed to overwrite existing data. The producer waits for space or uses a secondary buffer when possible.
- Buffer-empty condition: The consumer waits until data becomes available instead of attempting to read invalid data.
- Buffer release: Whenever data is consumed, count is decremented and the freed buffer space becomes available for new data.
- Dynamic monitoring: Occupancy thresholds can be used to give higher processing priority to a buffer that is becoming full.
- Data-loss prevention: Circular-buffer management, waiting, and optional secondary storage prevent

overwriting valid data during temporary bursts.

This mechanism is particularly useful when I/O devices operate at different and varying data rates, because the buffers absorb short-term differences between producer and consumer speeds.

1. low-priority devices from being indefinitely delayed.

4 Pseudocode: Priority-Based Resource Allocation

CONSTANT MAX_RESOURCES = R

CONSTANT AGING_RATE = A

CONSTANT PRIORITY_MAX = P_MAX

STRUCT DEVICE:

```
id
base_priority
dynamic_priority
bandwidth_required
bandwidth_allocated
data_pending
waiting_time
status      // WAITING, ACTIVE, COMPLETED
```

GLOBAL:

```
devices[]
available_resources
resource_owner[]
```

INITIALIZE():

```
available_resources = MAX_RESOURCES
```

```
for each device in devices do
    device.dynamic_priority = device.base_priority
    device.bandwidth_allocated = 0
    device.waiting_time = 0
    device.status = WAITING
end for
```

// Periodically update priorities

UPDATE_PRIORITIES():

```
for each device in devices do

    if device.data_pending > 0 then

        // Aging prevents indefinite starvation
        device.dynamic_priority =
            device.base_priority +
            (device.waiting_time * AGING_RATE)

        if device.dynamic_priority > PRIORITY_MAX then
            device.dynamic_priority = PRIORITY_MAX
        end if

    else
        device.waiting_time = 0
        device.dynamic_priority = device.base_priority
    end if

end for
```

```
// Select the highest-priority eligible device  
SELECT_DEVICE():
```

```
    selected = NULL  
  
    for each device in devices do  
  
        if device.data_pending > 0 AND  
           device.status == WAITING AND  
           device.bandwidth_required <= available_resources then  
  
            if selected == NULL OR  
               device.dynamic_priority > selected.dynamic_priority then  
  
                selected = device  
            end if  
        end if  
  
    end for  
  
    return selected
```

```
// Allocate resources and start transmission  
ALLOCATE_RESOURCE():
```

```
    UPDATE_PRIORITIES()  
  
    while available_resources > 0 do  
  
        device = SELECT_DEVICE()  
  
        if device == NULL then  
            break  
        end if  
  
        allocation =  
            MIN(device.bandwidth_required,  
                available_resources)  
  
        device.bandwidth_allocated = allocation  
        available_resources =  
            available_resources - allocation  
  
        device.status = ACTIVE  
  
        RESOURCE_ASSIGN(device, allocation)  
  
        START_TRANSMISSION(device)  
    end while
```

```
// Called when a device finishes transmitting  
TRANSMISSION_COMPLETE(device):
```

```
    STOP_TRANSMISSION(device)  
  
    released = device.bandwidth_allocated  
  
    available_resources =
```

```

    available_resources + released

device.bandwidth_allocated = 0
device.data_pending = 0
device.waiting_time = 0
device.status = COMPLETED

RESOURCE_RELEASE(device, released)

// Immediately make released resources available
ALLOCATE_RESOURCE()

// Called periodically while devices are waiting
RESOURCE_MANAGER():

    periodically do

        for each device in devices do

            if device.data_pending > 0 AND
               device.status == WAITING then

                device.waiting_time =
                    device.waiting_time + 1
            end if

        end for

        UPDATE_PRIORITIES()
        ALLOCATE_RESOURCE()

    end periodically

// New transmission request
REQUEST_RESOURCE(device, required_bandwidth):

    device.bandwidth_required = required_bandwidth
    device.data_pending = GET_PENDING_DATA(device)
    device.waiting_time = 0
    device.status = WAITING

    ALLOCATE_RESOURCE()

```

Key features

- **Priority-based allocation:** Devices with greater dynamic_priority are selected first, provided their bandwidth requirement can be satisfied.
- **Dynamic priority:** A device's priority increases as its waiting_time increases.
- **Starvation prevention:** The aging mechanism ensures that a low-priority device eventually becomes sufficiently urgent to receive resources.
- **Bandwidth awareness:** A device is eligible only when sufficient resources are available for its required bandwidth.
- **Resource release:** When transmission finishes, all resources allocated to that device are returned to the resource pool.
- **Continuous allocation:** Newly released resources are immediately offered to other eligible devices.
- **Fairness:** Priority aging balances urgency with fairness, preventing high-priority traffic from permanently blocking low-priority communication devices

1. **5** Pseudocode: Real-Time I/O Scheduling Mechanism
2. STRUCT IO_REQUEST:
3. device_id

```

4.   data_size
5.   transfer_duration
6.   deadline
7.   priority
8.   release_time
9.   remaining_time
10.  status      // READY, RUNNING, COMPLETED, MISSED
11.
12. GLOBAL:
13.   ready_queue
14.   pending_queue
15.   current_time
16.   available_bandwidth
17.   current_request
18.
19.
20. INITIALIZE():
21.   current_time = 0
22.   available_bandwidth = SYSTEM_BANDWIDTH
23.   ready_queue = empty priority queue
24.   pending_queue = empty list
25.   current_request = NULL
26.
27.
28. // Called when a peripheral requests a data transfer
29. ADD_REQUEST(request):
30.
31.   request.remaining_time = request.transfer_duration
32.   request.status = READY
33.
34.   INSERT ready_queue according to:
35.     1. Earliest deadline
36.     2. Higher priority
37.     3. Shorter remaining transfer time
38.
39.   SCHEDULE_TRANSFERS()
40.
41.
42. // Select the most urgent transfer
43. SELECT_REQUEST():
44.
45.   best_request = NULL
46.
47.   for each request in ready_queue do
48.
49.     if request.release_time <= current_time AND
50.        request.remaining_time <=
51.           AVAILABLE_TRANSFER_TIME(request) then
52.
53.       if best_request == NULL OR
54.          request.deadline < best_request.deadline OR
55.          (request.deadline == best_request.deadline AND
56.           request.priority > best_request.priority) then
57.
58.         best_request = request
59.       end if
60.     end if
61.   end for
62.
63.   return best_request

```

```

64.
65.
66. SCHEDULE_TRANSFERS():
67.
68.   while ready_queue is not empty do
69.
70.       request = SELECT_REQUEST()
71.
72.       if request == NULL then
73.           // No request can safely be completed now
74.           HANDLE_PENDING_REQUESTS()
75.           break
76.       end if
77.
78.       REMOVE request from ready_queue
79.
80.       request.status = RUNNING
81.       current_request = request
82.
83.       ALLOCATE_BANDWIDTH(request)
84.
85.       START_IO_TRANSFER(request.device_id,
86.           request.data_size)
87.
88.       while request.remaining_time > 0 do
89.
90.           EXECUTE_TRANSFER_FOR_TIME_SLICE()
91.
92.           request.remaining_time =
93.               request.remaining_time - TIME_SLICE
94.
95.           current_time =
96.               current_time + TIME_SLICE
97.
98.           MONITOR_DEADLINES()
99.
100.           // Allow an emergency/deadline-critical request
101.           // to pre-empt the current transfer if supported
102.           urgent = CHECK_FOR_URGENT_REQUEST()
103.
104.           if urgent != NULL AND
105.               urgent.deadline < request.deadline then
106.
107.               PAUSE_TRANSFER(request)
108.               request.status = READY
109.               INSERT request into ready_queue
110.
111.               current_request = NULL
112.               break
113.           end if
114.       end while
115.
116.       if request.remaining_time <= 0 then
117.
118.           STOP_IO_TRANSFER(request.device_id)
119.
120.           request.status = COMPLETED
121.           RELEASE_BANDWIDTH(request)
122.
123.           current_request = NULL

```

```

124.     end if
125. end while
126.
127.
128. MONITOR_DEADLINES():
129.
130.     for each request in ready_queue do
131.
132.         if current_time > request.deadline AND
133.            request.status != COMPLETED then
134.
135.             request.status = MISSED
136.             RECORD_MISSED_DEADLINE(request)
137.
138.             REMOVE request from ready_queue
139.             ADD request to pending_queue
140.         end if
141.     end for
142.
143.
144. HANDLE_PENDING_REQUESTS():
145.
146.     for each request in pending_queue do
147.
148.         if request.status == MISSED then
149.
150.             // Increase urgency of a missed request
151.             request.priority =
152.                 request.priority + DEADLINE_PENALTY
153.
154.             // Give the request a new scheduling deadline
155.             request.deadline =
156.                 current_time + RECOVERY_INTERVAL
157.
158.             request.status = READY
159.
160.             INSERT request into ready_queue
161.         end if
162.     end for
163.
164.     CLEAR pending_queue
165.
166.     // Recalculate scheduling order
167.     SORT ready_queue BY:
168.         earliest deadline,
169.         highest priority,
170.         shortest remaining time
171.
172.
173. AVAILABLE_TRANSFER_TIME(request):
174.
175.     return request.deadline - current_time
176.
177.
178. CHECK_FOR_URGENT_REQUEST():
179.
180.     urgent = NULL
181.
182.     for each request in ready_queue do
183.

```

```

184.         if request.release_time <= current_time AND
185.            request.deadline - current_time
186.                <= URGENT_THRESHOLD then
187.
188.            if urgent == NULL OR
189.               request.deadline < urgent.deadline then
190.
191.                urgent = request
192.            end if
193.        end if
194.    end for
195.
196.    return urgent
197.
198.
199. PERIODIC_SCHEDULER():
200.
201.     every SCHEDULER_PERIOD do
202.
203.         current_time = GET_SYSTEM_TIME()
204.
205.         MONITOR_DEADLINES()
206.
207.         HANDLE_PENDING_REQUESTS()
208.
209.         SCHEDULE_TRANSFERS()
210.
211.     end every

```

Scheduling strategy

- Earliest deadline first: Transfers with the closest deadlines are normally selected first.
- Priority handling: If two requests have similar deadlines, the higher-priority device is selected.
- Transfer duration: A request is scheduled only when its remaining transfer time can fit within its available deadline window.
- Bandwidth management: Each transfer receives the required system bandwidth, which is released after completion.
- Pre-emption: If a more urgent request arrives, the current transfer can be paused and resumed later.
- Deadline monitoring: The scheduler continuously checks for missed deadlines and records them.
- Rescheduling: Missed requests are placed in a pending queue, given increased urgency, and assigned a new recovery deadline.
- Responsiveness: Time-critical I/O is handled first while incomplete transfers remain eligible for later scheduling, improving overall real-time responsiveness.

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

Q. No	Problem Understanding (20)	Algorithm Design (30)	Use of Control Structures (20)	Pseudocode Structure (10)	Completeness (20)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/30	/ 20	/ 10	/ 20	/ 100

Course Coordinator

Faculty Incharge